

Data Modeling:

Data modeling is the process of designing how data is structured, stored, related and accessed in a database or data warehouse. It's the blueprint—deciding what tables exist, what columns they have, and how they connect.

Ex:

In e-commerce system.

Customer, Product, Order, OrderItem, Payment

Relationships:

Customer 1 — N Order

Order 1 — N OrderItem

Product 1 — N OrderItem

Order 1 — 1 Payment

This structure is the data model.

what happens without a Data Model?

1. Data redundancy
2. update anomalies
3. Insert anomalies
4. Delete anomalies
5. Difficult relationships
6. Poor performance
7. Poor data quality

A good model provides:

Consistency

+ Less redundancy

+ Clear relationships

+ Data integrity

+ Better performance

+ Easier maintenance

+ Easier analytics

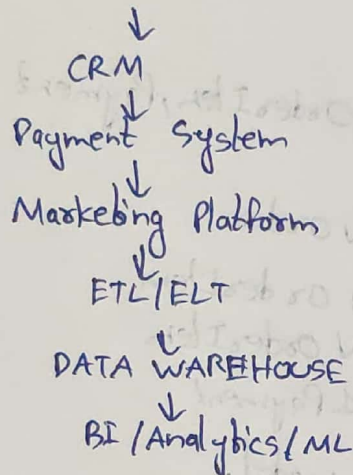
Data warehouse:

→ is a centralized system designed primarily for analytics, reporting, historical analytics, and decision-making.

It collects data from multiple sources.

Ex:

Application Database



Transactional Database vs Data warehouse

Transactional Database — designed to run the business

Data warehouse — designed to analyze the business.

OLTP vs OLAP

OLTP — Online Transaction Processing

→ OLTP systems handle day-to-day transactions.

Ex: Banking systems, E-commerce applications, Airline booking
Food delivery, user registration.

Typical operations: INSERT, UPDATE, DELETE, SELECT

OLAP — Online Analytical Processing

→ OLAP systems are designed for analytical queries.

Ex: `SELECT year, region, SUM (revenue) FROM sales GROUP BY year, region;`

The goal is: Fast analysis over large amounts of data.

Feature	OLTP	OLAP
Purpose	Run business	Analyze business
Data	Current operational data	Historical + integrated data
queries	small/simple	large/complex
operations	INSERT/UPDATE/DELETE	mostly SELECT
users	Applications/users	Analysts/data scientists
Data volume	usually smaller	very large
Schema	usually normalized	often denormalized
Transactions	very important	Less central
Response	Milliseconds	seconds/minutes depending on workload
Example	PostgreSQL application DB	snowflake/BigQuery/Redshift

Why shouldn't we run analytics directly on OLTP?

Production databases contain large amounts of data. An analyst query may require scanning huge amounts of data. At the same time other transactional operations happens.

The analytical query could compete for: CPU, memory, disk I/O database connections

This can slow down application.

Therefore: OLTP Database → ETL/ELT → Data warehouse → Analytics

The production database focuses on transactions, while the warehouse focuses on analytics.

Three levels of Data Modeling

1. Conceptual Model
2. Logical Model
3. Physical Model

Conceptual Data Model

→ gives a high-level view of the business.
It focuses on: major entities, major relationships,
business concepts.

It does not worry about detailed columns or database-specific implementation.

Ex: Customer $\xrightarrow{\text{places}}$ Order $\xrightarrow{\text{contains}}$ Product

Logical Data Model

→ adds more detail

defines: entities/tables, Attributes, Relationships/Primary keys,
Foreign keys, Cardinality, constraints, Normalization.

Ex:

CUSTOMER

CustomerID PK
Name
Email
Phone

ORDER

OrderID PK
CustomerID FK
OrderDate

PRODUCT

ProductID PK
ProductName
Price

Relationships:

CUSTOMER 1 — N ORDER

ORDER N — N PRODUCT

many-to-many relationship may be resolved using:

ORDERITEM

OrderID FK
ProductID FK
Quantity

Physical Data Model

→ describes how the model will actually be implemented in a specific database system.

it includes things such as: exact data types, indexes, partitions, constraints, storage details, Database-specific features.

Ex PostgreSQL:

```
CREATE TABLE Customer(  
  Customer-id BIGINT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(255) UNIQUE  
);
```

Normalization:

→ is the process of organizing data to:

Reduce redundancy, prevent data anomalies, improve data integrity,
ensure each piece of information has an appropriate place.

Ex:

Non-Normalized

OrderID	Customer	CustomerPhone	Product	Price
101	Ravi	9999	iPhone	70000
102	Ravi	9999	Charger	2000
103	Ravi	9999	AirPods	15000

Customer information is repeated

Normalized

Customer

CustomerID	Name	Phone
1	Ravi	9999

Product

ProductID	Product	Price
10	iPhone	70000
11	Charger	2000
12	AirPods	15000

Order

OrderID	CustomerID	Order Date
101	1	2026-08-01
102	1	2026-08-02

Now customer information is stored once.

Normalization helps prevent three classic anomalies:

update anomaly, insert anomaly, delete anomaly

Normal Forms: 1NF, 2NF, 3NF, BCNF, 4NF, 5NF

1NF - First Normal Form:

A table is in 1NF when:

- Each column contains atomic values
- There are no repeating groups
- Each row represents a distinct record.

Bad:

StudentID	Name	PhoneNumbers
1	Ravi	9999, 8888

Better:

StudentID	Name	Phone
1	Ravi	9999
1	Ravi	8888

2NF - Second Normal Form:

A table must:

- be in 1NF
- have no partial dependency on part of a composite key.

Ex: OrderID, ProductID, ProductName, Quantity

Suppose: PK = (OrderID, ProductID)

if ProductID $\rightarrow$ ProductName

then ProductName depends only on ProductID, not the complete composite key.

That is partial dependency

Solution:

move product information into:

PRODUCT
ProductID
ProductName

3NF - Third Normal Form

A table must be:

- in 2NF
- have no transitive dependency

Example:

EmployeeID, EmployeeName, DepartmentID, DepartmentName

Dependencies:

EmployeeID $\rightarrow$ DepartmentID

DepartmentID $\rightarrow$ DepartmentName

Therefore: EmployeeID $\rightarrow$ DepartmentName indirectly

That is a transitive dependency

Separate:

EMPLOYEE

EmployeeID

EmployeeName

DepartmentID

DEPARTMENT

DepartmentID

DepartmentName

Why Analytics Deliberately Denormalizes

OLTP and OLAP have different goals.

OLTP wants Normalization

OLTP wants: Data integrity + Less duplication + Efficient updates
+ Transactional consistency

Therefore: Customer $\rightarrow$ Order $\rightarrow$ OrderItem $\rightarrow$ Product

may be represented through multiple normalized viewables.

OLAP wants Denormalization

Analytics wants: Fast reads + simple queries + Large-scale aggregations
+ Fewer joins

Ex:

Suppose an analyst wants:

"Total sales by product category and month".

In a highly normalized schema, they may need:

Sales $\rightarrow$ Order $\rightarrow$ Customer $\rightarrow$ Product $\rightarrow$ Category $\rightarrow$ Date

several joins may be required.

A dimensional warehouse might instead provide:

FACT-SALES

Datekey
Productkey
Customerkey
Storekey
Quantity
SalesAmount

DIM-PRODUCT

ProductKey
ProductName
Category
Brand

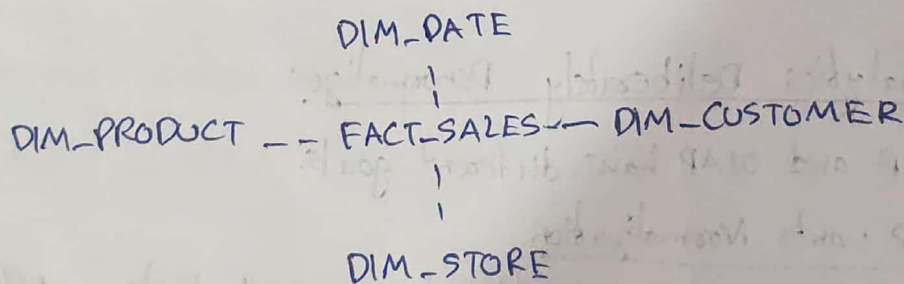
DIM-DATE

Datekey
Date
Month
Quarter
Year

This is designed specifically for analytical queries.

Star Schema

structure:



The centre is the **FACT-TABLE**, around it are **DIMENSION TABLES**.

Fact Table

→ stores measurable business events.

Ex: sales, orders, payments, clicks, transactions, shipments

FACT - SALES

Datekey
Productkey
Customerkey
Storekey
Quantity
SalesAmount
DiscountAmount

measures include:

Quantity
Revenue
Cost
Profit
Discount

Dimension Table

→ describes the context around facts

Ex:

DIM-CUSTOMER
DIM-PRODUCT
DIM-DATE
DIM-STORE
DIM-EMPLOYEE

DIM-PRODUCT

Productkey
ProductName
Brand
Category
Subcategory

Facts answer: How much? How many?

Dimensions answer: who? what? where? when?

Natural key:

→ is a key that comes from the real-world/business data.

Ex:

Email
Phone Number
ISBN
PAN
Product SKU
Government ID

PRODUCT

SKU
ProductName
Price

SKU could be natural key, because SKU already exists as part of the business.

Surrogate key

→ is an artificial key created specifically for the database

Ex:

Productkey

1001

1002

1003

It has no business meaning.

PRODUCT

Productkey	SKU	Product Name
1001	IP15	iPhone 15
1002	CH100	Charger

here:

Productkey - surrogate key

SKU → Natural/business key

Feature	Natural key	Surrogate key
Comes from business data	Yes	No
artificially generated	No	Yes
has business meaning	usually	No
Can change	Potentially	usually doesn't
often composite	Sometimes	usually simple
Good for warehouse joins	less ideal	usually preferred
EX	SKU	Product key = 1001

why use surrogate keys:

1. Business values can change
2. Natural keys can be composite
3. Natural keys can be large
4. Natural keys can be reused
5. Historical tracking

Natural keys are used for:

- Business validation
- Deduplication
- Data integration
- Matching incoming data
- Auditing

Data Modeling in a Real Company

typical flows:

BUSINESS REQUIREMENTS



CONCEPTUAL MODEL



LOGICAL MODEL



PHYSICAL MODEL



DATABASE / WAREHOUSE

For analytics:

Operational Systems



ETL / ELT



staging



Data Warehouse



Dimensional Model



BI / Reporting / ML

Kimball Methodology

→ is a bottom-up approach to Data warehouse design that builds the warehouse using business process-oriented data marts.

identify business processes → define grain → create fact tables →
add dimension tables → integrate them using conformed dimensions.
Kimball mainly uses a star schema.

Fact Table

→ stores measurable business events/measures and foreign keys pointing to dimensions

Ex: sales-fact

date-key	product-key	customer-key	store-key	quantity	sales-amount
101	501	1001	201	2	2000

Contains:

- Foreign keys → dimensions
- Numeric measurements → facts

Typical facts:

• quantity • revenue • profit • discount • cost

Simple Structure:

```
CREATE TABLE sales-fact(  
    date-key INT,  
    product-key INT,  
    customer-key INT,  
    quantity INT,  
    sales-amount DECIMAL (10,2)  
);
```


Dimension Table

→ stores the descriptive/concept contextual information used to analyze facts.

Ex: product-dim

Product-key	product-name	category	brand
301	iphone 14	mobile	Apple

dimensions answer: who? what? where? when? why?

Ex:

```
CREATE TABLE product_dim(  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category VARCHAR(50),  
    brand VARCHAR(50)  
);
```

Fact Table	Dimension Table
stores business events	stores descriptions
mostly numeric data	mostly textual/categorical data
contains foreign keys	contains primary / surrogate key
usually very large	usually smaller
used for calculations	used for filtering/grouping
Ex: sales-amount	Ex: product-name

Ex: Fact = what happened? Dimension = describe what happened?

Customer bought 2 iPhones for ₹ 2000

2 → Fact

2000 → Fact

iPhone → Product Dimension

Customer → Customer Dimension

Date → Date Dimension

Grain:

→ defines exactly what one row in a fact table represents.

Ex:

if grain is:

one row = one product on one order

Then

order 1001 + iPhone = 1 row

order 1001 + Laptop = 1 row

Ex Fact:

order-id	product	quantity	amount
1001	iPhone	2	2000
1001	Laptop	1	80000

*define grain BEFORE deciding the facts and dimensions.
bad design:

"sales table contains sales information"

good design:

"each row represents one product line purchased in an order".

Types of Fact Tables

A. Transaction Fact Table

stores individual business transactions/events.

Ex:

one row = one product purchased in one order.

Ex: sales, orders, Payments, Transactions:

order 1001 → iPhone → 50,000

order 1001 → AirPods → 20,000

most common type.

B. Periodic Snapshot Fact Table

→ stores the state of a business at regular intervals.

Ex:

one row = inventory status for one product one day

date	product	stock
Jan 1	iPhone	100
Jan 2	iPhone	80
Jan 3	iPhone	75

used for:

- Daily Inventory, • Monthly account balance, • Daily bank balance.

C. Accumulating Snapshot Fact Table

Tracks a process from beginning to end.

Ex: order fulfillment

Order Created → Payment → Packing → Shipping → Delivery

one row represents one order/process.

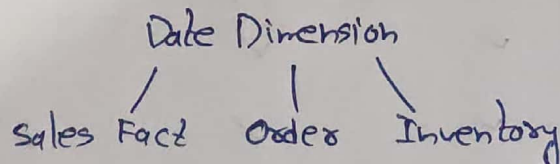
order | order-date | payment-date | ship-date | delivery-date

used when a process has multiple milestones

Conformed/Conformed Dimensions:

→ is a dimension that is shared consistently across multiple fact tables/business processes.

Ex:



All processes use the same: date-dim

Ex: date-dim can be used by: sales, orders, Inventory Payments.

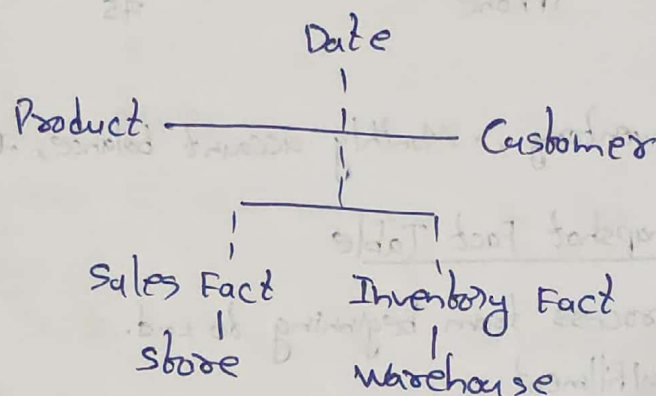
conformed dimension: common, standardized dimension shared across fact tables.

why imp? it allows consistent analysis

Kimball Bus Architecture

→ provides a standard framework for integrating multiple business processes using conformed dimensions.

Think of it like a bus:



The dimensions are the common bus, and different fact tables connect to them.

Bus Matrix:

A common planning tool is the Bus Matrix

Business Process	Date	Product	Customer	Store
Sales	✓	✓	✓	✓
Inventory	✓	✓		✓
Returns	✓	✓	✓	✓

This helps identify:

- which dimensions are shared
- which dimensions are needed
- How different data marts integrate.

Special Dimensions:

A. Degenerate Dimension

→ is a dimension attribute stored directly inside the fact table because it has no separate dimension table.

usually identifiers such as: Order number, Invoice number
Ticket number, Transaction number.

Ex: sales-fact

order-number
Product-key
Customer-key
quantity
sales-amount

order-number is a degenerate dimension.

Identifier + no separate dimension = Degenerate Dimension

B. Junk Dimension

→ combines small, unrelated, low-cardinality attributes into one dimension.

Instead of putting many small flags in the fact table:

is-gift, is-online, is-first-order, payment-type

Create:

→ junk-dim

Ex:

junk-key	is-gift	is-online	is-first-order	payment-type
1	Y	Y	N	Card
2	N	Y	Y	UPI
3	N	N	N	Cash

Fact table:

sales-fact

Product-key
Customer-key
junk-key

sales-amount

→ keeps the fact table cleaner and avoids creating many tiny dimensions.

small miscellaneous flags / attributes → Junk Dimension.

C. Role-playing Dimension:

→ is the same physical dimension used multiple times for different business roles.

Ex: Date Dimension

An order can have:

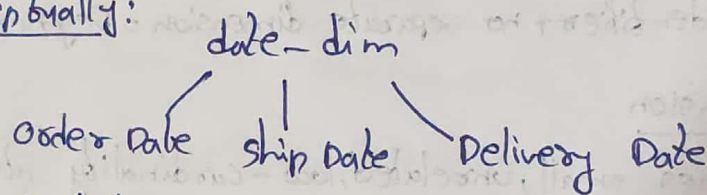
order-date

ship-date

delivery-date

All three can use the same: date-dim

conceptually:



In a fact table:

order-fact

order-date-key

ship-date-key

delivery-date-key

All keys reference date-dim.

same dimension + different meaning/role = Role-playing dimension

Slowly Changing Dimensions (SCD)

→ is a technique used in a data warehouse to manage changes in dimension data over time.

SCD Type 0 - keep original value

→ once the dimension value is stored, it is never changed

use when: the original value is important and should never change.

Ex: Customer's original registration date, DOB, original customer category.

```
UPDATE dim-customer SET city = 'Bangalore' WHERE  
customer-id = 101;
```

Type 0 would not perform this update

SCD Type 1 - overwrite

→ update the existing dimension record with the new value.
history is not preserved

Ex:

customer-id	name	city
101	Ravi	Hyderabad

After change:

customer-id	name	city
101	Ravi	Bangalore

```
UPDATE dim-customer SET city = 'Bangalore' WHERE  
customer-id = 101;
```

use when:

- historical value is not important
- Data correction is required
- You only care about the current value

SCD Type 2 - Maintain Full History

→ when a dimension attribute changes, close the old record and insert a new record.

we preserve history.

Typical columns:

customer-key

customer-id

name

city

start-date

end-date

is-current

Ex:

customer-key	customer-id	name	city	start-date	end-date
1	101	Ravi	Hyderabad	2025-01-01	2026-08-20
2	101	Ravi	Bangalore	2026-08-21	NULL

customer-id = business/natural key

customer-key = warehouse surrogate key

This allows the same customer to have multiple historical records.

SCD Type 3 - keep limited History

→ store the current value and previous value in the same row.

Ex:

customer-id	current-city	previous-city
101	Bangalore	Hyderabad

when another change happens: Bangalore → Chennai becomes:

customer-id	current-city	previous-city
101	Chennai	Bangalore

Hyderabad is lost.

use when: you only need limited history.

SCD type 6 - Hybrid SCD

→ combines Type 1, 2, 3

It maintains historical rows like Type 2, while also maintaining a current value column like Type 3/Type 1.

Ex:

customer-key	customer-id	city	previous-city	current-city
1	101	Hyderabad	Hyderabad	Bangalore
2	101	Bangalore	Hyderabad	Bangalore

when customer moves to Chennai:

customer-key	customer-id	city	previous-city	current-city
1	101	Hyderabad	Hyderabad	Chennai
2	101	Bangalore	Hyderabad	Chennai
3	101	Chennai	Bangalore	Chennai

The historical city is preserved, while current-city represents the latest value.

use when: you need full history, previous value, easy access to current value.

How to choose SCD Type per Attribute

you don't necessarily choose one SCD type for the entire dimension. you can choose it attribute by attribute.

Should the value NEVER change? Type 0

Do I only care about the latest value? Type 1

Do I need complete history? Type 2

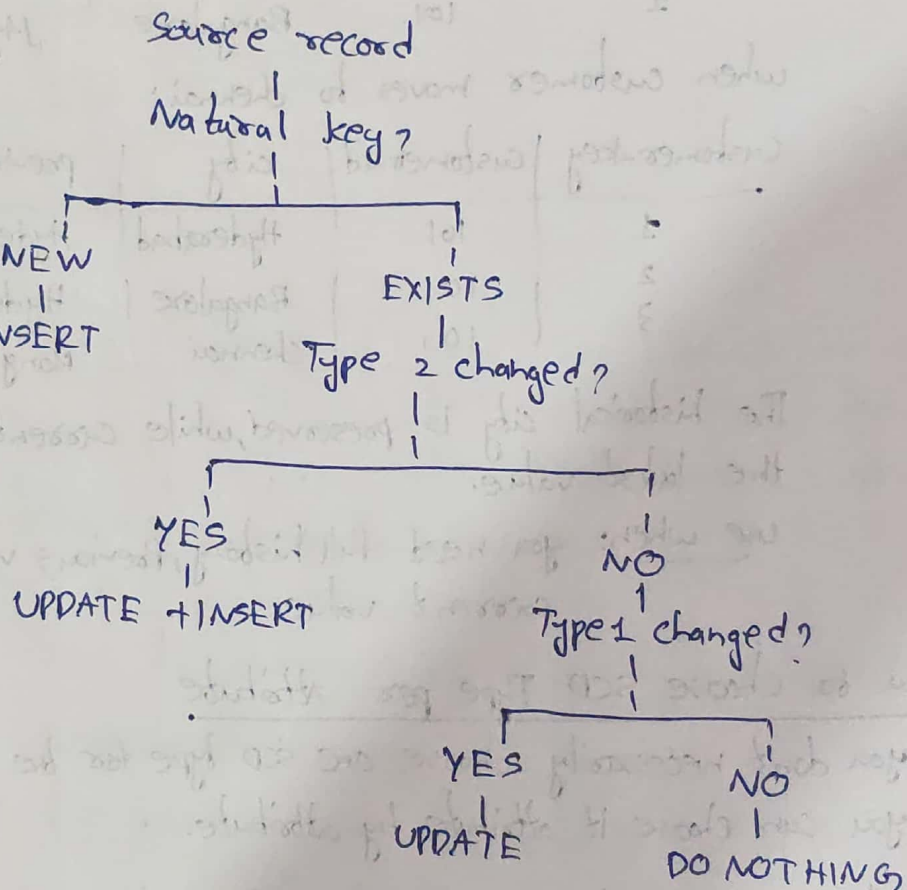
Do I only need current + previous value? Type 3

Do I need full history + current/previous convenience columns? Type 6.

Implementing Type 2 in an ETL pipeline:

- i. New natural key $\rightarrow$ INSERT
- ii. Existing key + no change $\rightarrow$ DO NOTHING
- iii. Existing key + tracked attribute changed (Type 2) $\rightarrow$ UPDATE + INSERT
- iv. Existing key + only Type 1 attribute changed $\rightarrow$ UPDATE in place

decision tree:



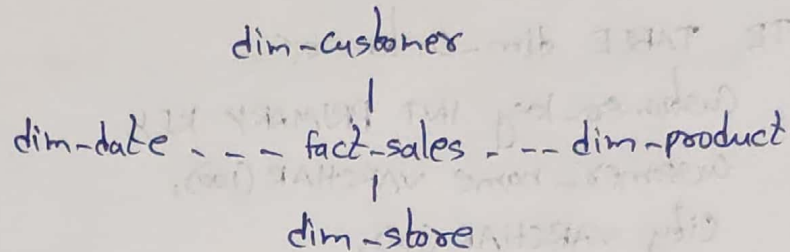
* modern warehouses support MERGE with WHEN MATCHED and WHEN NOT MATCHED. But Type 2's "close + insert" is really two writes, so most ETL frameworks split it into an UPDATE step (close old rows) followed by an INSERT step (add new versions).

Schema

→ logical structure/organization of tables, columns, relationships, and keys in a database/data warehouse.

In a data warehouse, schema defines how fact tables and dimension tables are organized.

Ex:



a good schema should make:

- data easy to query
- Relationships clear
- Analytics fast
- Data deduplication manageable

Common Schema Patterns:

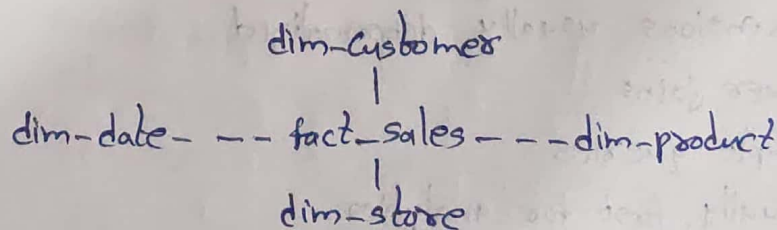
main dimensional modeling patterns:

1. star schema
2. snowflake schema
3. Galaxy schema/Fact constellation.

Star Schema

→ a star schema has one central fact table connected directly to multiple denormalized dimension tables.

Ex:



Ex tables:

```
CREATE TABLE dim-product(  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category VARCHAR(100),  
    brand VARCHAR(100)  
);
```

```
CREATE TABLE dim-customer(  
    customer-key INT PRIMARY KEY,  
    customer-name VARCHAR(100),  
    city VARCHAR(100),  
    state VARCHAR(100)  
);
```

```
CREATE TABLE fact-sales(  
    sales-key BIGINT PRIMARY KEY,  
    date-key INT,  
    product-key INT,  
    customer-key INT,  
    quantity INT,  
    sales-amount DECIMAL(10, 2)  
);
```

characteristics:

- Fact table at center
- Dimensions directly connected to fact
- Dimensions usually denormalized
- Fewer joins
- Simple queries
- Usually fast for analytics

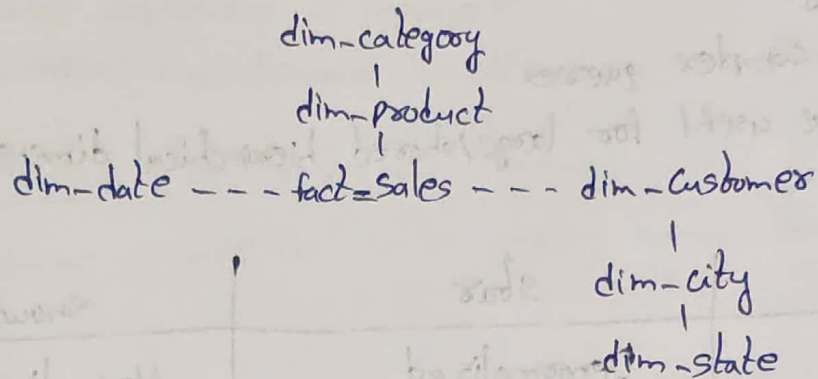
Ex query:

```
SELECT p.category, SUM(f.sales-amount) AS total-sales FROM  
fact-sales f JOIN dim-product p ON fproduct-key=pproduct-key  
GROUP BY p.category;
```


Snowflake schema

→ is a normalized version of a star schema where dimensions are split into multiple related tables.

Ex:



instead of:

dim-product
product-key
product-name
category
brand

we normalize:

dim product
product-key
product-name
category-key
brand-key

dim-category
category-key
category-name

SQL:

```
CREATE TABLE dim-category(  
    category-key INT PRIMARY KEY,  
    category-name VARCHAR(100)  
);
```

```
CREATE TABLE dim-product(  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category-key INT,  
    FOREIGN KEY (category-key) REFERENCES  
    dim-category(category-key)  
);
```

query:

```
SELECT c.category-name, sum(f-sales-amount) FROM fact-sales f  
JOIN dim-product p ON f-product-key = p.product-key JOIN  
dim-category c ON p.category-key = c.category-key  
GROUP BY c.category-name;
```

Characteristics:

- Dimensions are normalized
- Less duplicate data
- More tables
- More joins
- More complex queries
- Can be useful for large/shared hierarchical dimensions.

Feature	star	Snowflake
Dimension	Denormalized	Normalized
Number of tables	Fewer	More
Joins	Fewer	More
Query simplicity	High	Lower
Storage duplication	Higher	Lower
Maintenance	Simple	More complex
BI usability	Excellent	More complex
Typical choice	Default	Specific situations

When to choose star:

- Analytics/query simplicity is important
- BI users frequently query the warehouse
- Dimensions aren't extremely large
- you want fewer joins
- Performance and usability are priorities

start with star schema unless you have a good reason to normalize dimensions.

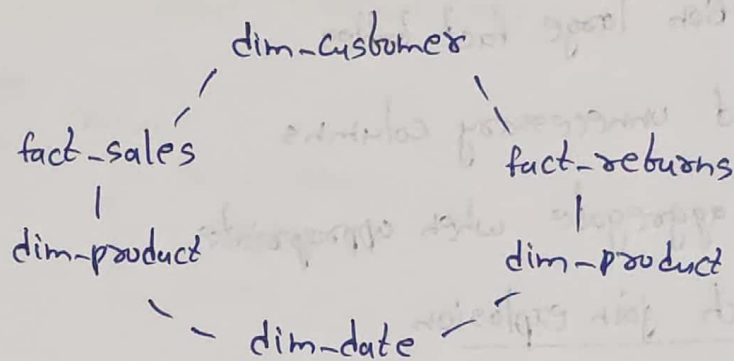
when to choose Snowflake

- Dimensions are very large
- Hierarchies are complex
- Dimension attributes are heavily shared
- Reduce duplication is important
- Dimension maintenance benefits from normalization

Galaxy Schema (Fact Constellation Schema)

→ contains multiple fact tables sharing common dimensions.

Ex:



Ex facts:

fact-sales

date-key
product-key
customer-key
quantity
sales-amount

fact-returns

date-key
product-key
customer-key
return-quantity
refund-amount

both facts share: • dim-date • dim-product • dim-customer

These are called conformed dimensions.

when to use?

use galaxy when the business has multiple business processes.

Ex: sales Orders Returns Inventory Payments Shipments
and these processes need common dimensions.

Performance Considerations:

Star schema:

usually performs well because: Fact $\rightarrow$ Dimension
requires fewer joins.

1. Index/clustering

depending on the database/warehouse, optimize frequently filtered or joined columns.

EX: CREATE INDEX intsales-product ON fact-sales(product-key);

2. Partition large fact tables

3. Avoid unnecessary columns

4. Pre-aggregate when appropriate

5. Watch join explosion

Joining multiple large fact tables directly can create incorrect results and huge intermediate datasets.

Common Modeling Pitfalls:

i. wrong grain

ii. over-normalizing dimensions

iii. under-normalizing everything

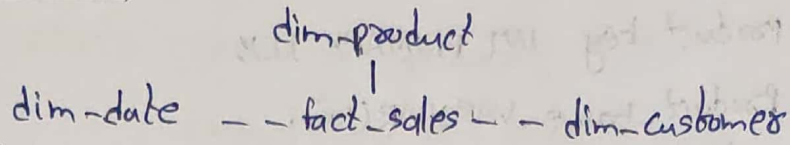
iv. Missing surrogate keys

v. Mixing grains

vi. Too many joins.

Star → Snowflake

Star schema:



Star version:

```
CREATE TABLE dim-product(  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category VARCHAR(100),  
    brand VARCHAR(100)  
);
```

here:

Product
├ Product Name
├ Category
└ Brand

Step 1: Snowflake the product dimension

create separate category table:

```
CREATE TABLE dim-category(  
    category-key INT PRIMARY KEY,  
    category-name VARCHAR(100)  
);
```

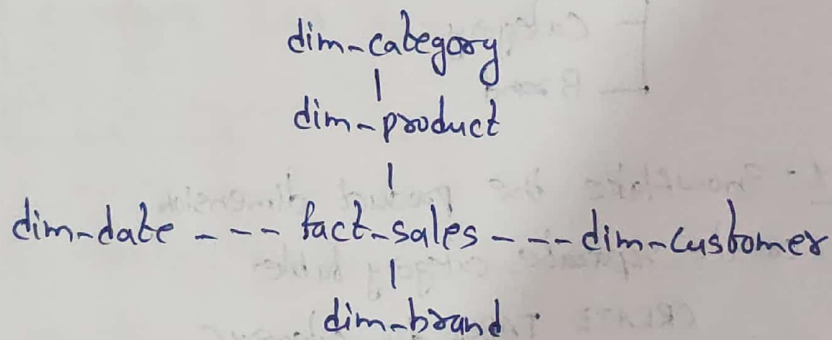
create brand table:

```
CREATE TABLE dim-brand(  
    brand-key INT PRIMARY KEY,  
    brand-name VARCHAR(100)  
);
```

change product dimension:

```
CREATE TABLE dim-product(  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category-key INT,  
    brand-key INT,  
    FOREIGN KEY (category-key) REFERENCES dim-category  
    (category-key),  
    FOREIGN KEY (brand-key) REFERENCES dim-brand  
    (brand-key)  
);
```

Now:



This is snowflake schema.

Snowflake → star

SQL:

```
CREATE TABLE dim-product (  
    product-key INT PRIMARY KEY,  
    product-name VARCHAR(100),  
    category VARCHAR(100),  
    brand VARCHAR(100).  
);
```

Instead of: fact-sales → dim-product → dim-category

we have: fact-sales → dim-product

Now the schema is again a star schema.

Query becomes simpler:

Snowflake

```
SELECT c.category-name, SUM(f.sales-amount) FROM  
fact-sales f JOIN dim-product p ON f.product-key =  
p.product-key JOIN dim-category c ON p.category-key =  
c.category-key GROUP BY c.category-name;
```

star

```
SELECT p.category, SUM(f.sales-amount) FROM fact-sales f  
JOIN dim-product p ON f.product-key = p.product-key  
GROUP BY p.category;
```

STAR

Fact → Dimension
↓
attributes

SNOWFLAKE

Fact → Dimension → Sub-dimension
↓
attributes.

Data Warehouse:

→ is a centralized system that stores cleaned, structured, historical data for analytics and reporting.

Typical flow:

Sources → ETL/ELT → Data Warehouse → BI/Reports/Dashboards

Ex:

Orders → Customers → Products → Sales

↓
Data Warehouse

↓
Power BI/ Tableau

where it shines:

- Fast analytical queries
- highly structured data
- Business reporting and BI
- Strong governance and security
- Consistent, trusted data
- SQL-friendly

Limitations

- less flexible for raw/unstructured data
- storage can be expensive
- schema usually needs to be designed before loading
- Large-scale ML/data science workloads can be less convenient.

Data Lake

→ stores raw data in its original form, usually on cheap object storage such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage

Sources → Data Lake → Process when needed → Analytics / ML / BI

It can store: CSV, JSON, Parquet, Images, Videos, Logs, IoT data, Application events

Schema-on-read

The data is stored first, its structure is interpreted when you read/process it.

Raw Data → Store → Apply Schema → Read/Analyze

Ex:

```
df = spark.read.json("s3://data/events/")
```

The schema can be applied during processing rather than requiring everything to be structured before storage.

where it shines:

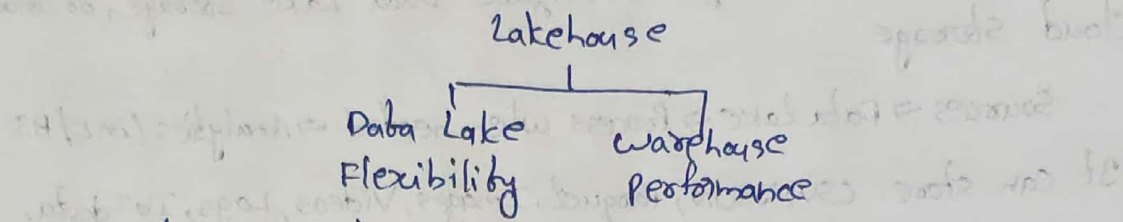
- Huge volumes of data
- Raw/unstructured/semi-structured data
- Data science and ML
- Flexible exploration
- cheap storage

Limitations

- Governance can become difficult
- Poorly managed lakes can become data swamps
- query performance may require additional processing/optimization.
- Data quality is often inconsistent

Lakehouse

→ combines the flexibility of a data lake with many capabilities of a warehouse.

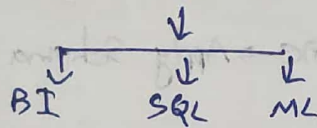


Typical architecture:

Applications / APIs / IoT

↓
Object Storage

↓
Open Table Format



It keeps data on relatively inexpensive object storage but adds:

- ACID transactions
- Schema enforcement
- Schema evolution
- Versioning / time travel
- Better data management
- SQL analytics

	Data warehouse	Data Lake	Lakehouse
Data	Structured	Any type	Any type
Schema	usually before load	on read	Flexible + governed
Storage	warehouse storage	object storage	object storage
Cost	higher	Low storage	low storage
BI	Excellent	Moderate	Excellent
ML	Moderate	Excellent	Excellent
Governance	strong	can be difficult	stronger
Flexibility	Medium	very high	very high
Performance	very high	variable	high

Main Trade-offs:

Governance:

warehouse → strong
 Lake → weaker by default
 Lakehouse → strong + flexible

Performance:

warehouse → optimized for analytics
 Lake → may require more processing before queries become efficient
 Lakehouse → improve this through table formats, partitioning, metadata, caching, file optimization etc.

Flexibility:

warehouse → low
 Lake → high
 Lakehouse → high

Cost Dimensions

i. Storage cost

Data Lake / Lakehouse

object storage

usually cheap per GB

ii. Compute cost

more data processing = more compute cost.

iii. Query cost

query only used required instead of everything

Open Table Formats

Object storage normally stores files:

s3
- file1.parquet
- file2.parquet

An open table format adds table-management capabilities on top of those files.

Major Formats:

i. Delta Lake:

Provides: ACID transactions, schema enforcement, schema evolution, Time travel, updates/deletes.

Ex:

```
SELECT * FROM sales VERSION AS OF 10;
```

ii. Apache Iceberg:

designed for large analytical datasets.

Provides: ACID transactions, schema evolution, partition evolution, Time travel, Hidden Partitioning

Ex:

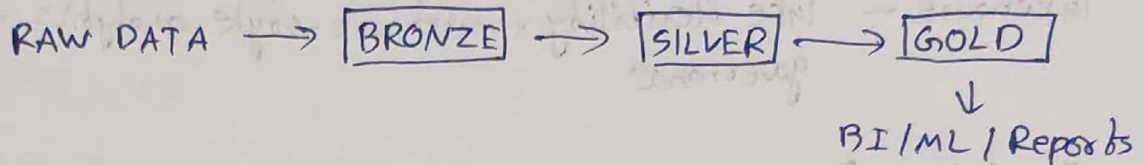
```
SELECT * FROM sales FOR SYSTEM_TIME AS OF  
TIMESTAMP '2026-08-01 00:00:00';
```


Apache Hudi

→ strong at streaming and incremental updates and frequently changing datasets.

useful for: CDC, upserts, streaming ingestion, Near-real-time data

Medallion Architecture



Bronze - Raw

stores data close to the source

API → Bronze CSV → Bronze logs → Bronze IoT → Bronze

Ex:

bronze/orders/

bronze/customers/

minimal transformation.

Silver - cleaned

clean and standardize the data

Typical operations:

```
SELECT customer-id, LOWER(email) AS email,  
CAST(order-date AS DATE) AS order-date  
FROM bronze-orders WHERE customer-id IS  
NOT NULL;
```

silver handles: • cleaning, deduplication, Type conversion,
data validation, joining related data.

Gold - Business-ready

data is transformed for specific business use cases.

Ex:

```
CREATE TABLE gold-daily-sales AS SELECT  
order-date, SUM(amount) AS total-sales, COUNT(*)  
AS total-orders FROM silver-orders GROUP BY  
order-date;
```

used by: BI dashboards Reports Business analysts, ML features

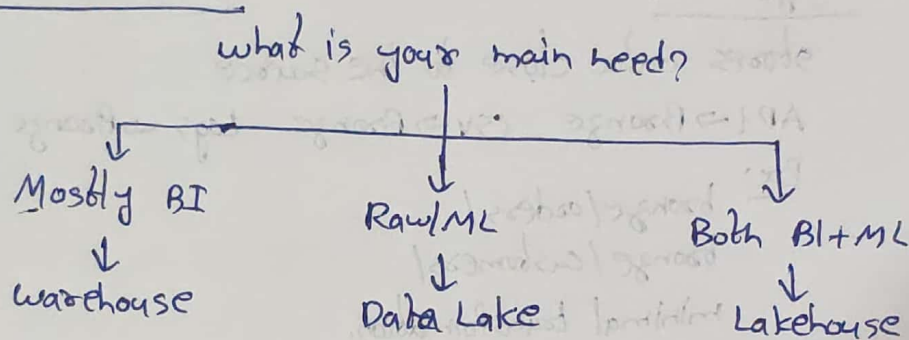
Data warehouse vs Lake vs Lakehouse

warehouse - reliable business reports quickly

data lake - store everything cheaply and decide later how to use it

lakehouse - take flexibility + warehouse-style analytics and governance

Decision Framework:



choose Data warehouse when:

- mostly structured data
- BI/reporting is the priority
- Strong governance is important
- Predictable SQL workloads
- Business users are the main consumers.

choose Data Lake when:

- massive raw data
- Data science/ML is important
- Many different data types
- Low-cost storage is a priority
- Future use cases are uncertain.

choose Lakehouse when:

- you need both BI+ML
- You want object-storage economics
- you need ACID transactions
- you need schema evolution
- you want one platform for many workloads.

Feature store

→ is a centralized system for storing, managing, computing and serving ML features consistently for both training and prediction.

why ML teams need one

without a Feature store:

Data Engineer → features for training

ML Engineer → different features for serving

Data Scientist → another calculation.

This causes:

- duplicated feature engineering
- inconsistent definitions
- train/serve skew
- difficult feature reuse
- difficult feature versioning
- poor data freshness management

Feature store gives:

Raw Data



Feature Engineering



Feature store



Offline store → Training



Online store → Real-time Prediction

Online vs Offline store Feature store

offline store

→ used mainly for training and historical analysis.
stores large amounts of historical feature data

Feature values

Customer-id	timestamp	avg-spend-7d
101	2021-08-20	2400
101	2021-08-21	2600

Typical storage: Data Warehouse, Data Lake, Parquet, Big Query, Snowflake

Online store

→ used for low-latency inference

usually key-value storage:

GET customer: 101

{ "avg-spend-7d": 2600,

"transaction-24h": 8

Typical technologies: Redis, DynamoDB, Cassandra

offline

Training

Historical

Huge data

seconds/minutes
acceptable

Online

Prediction

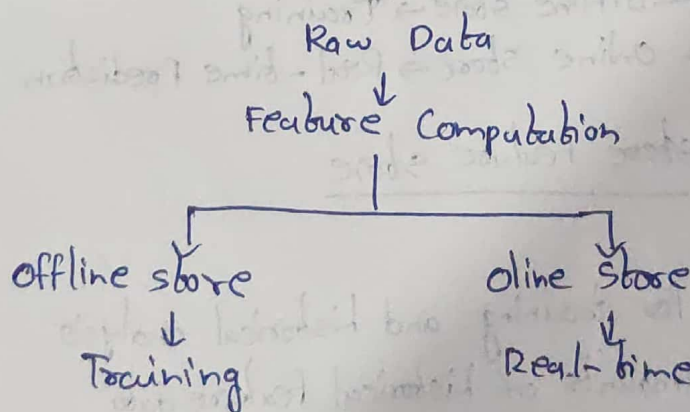
Latest/current

smaller

milliseconds required

Dual-Store Architecture:

most production Feature stores use both



Point-in-Time Correctness

→ means that when you train a model on historical labels, the features joined to each label reflect only what was known at that label's timestamp - never future data. Getting this wrong is called data leakage, and it produces models that look perfect in training and fail in production.

Feature stores help automatically retrieve the feature values that existed at that historical timestamp.

The Sync flow (called materialization) is

- i. Feature computation - a batch or streaming job computes features from raw data.
- ii. write to offline store - historical rows appended (large volume, time-indexed)
- iii. Publish latest to online store - only the current row per entity written to the low-latency store.
- iv. Both stores read the same definition - no ad-hoc feature logic anywhere else.
- v. Idempotent + versioned - same materialization run twice = same result.

Train - Serve skew

→ occurs when the feature calculated during training differs from the feature calculated during production inference.

Three causes of skew:

- i. Definition drift - training uses a spark job, serving uses a hand-rolled SQL query. subtle logic differences appear.
- ii. Time drift - training uses batch nightly data, serving uses real-time streaming. same feature name, different freshness
- iii. population drift - training data was filtered, production sees everyone. Feature distributions shift.

How feature store prevent skew?

1. single feature definition - one canonical spec, written once
2. Materialization to both stores - same code writes offline and online.
3. Consistency tests - periodically sample entities and verify offline value == online value
4. Monitoring - track live feature distributions vs training distributions
5. Shadow serving - before deploying, run new features in parallel and compare.

Entity - Based Feature Design:

An entity is the thing the feature describes

Ex: Customer, Product, Driver

Ex: Entity: customer-id

Features:

customer-id

total-orders-30d

avg-order-value

failed-payments-7d

Feature Groups

→ is a logical collection of related features

Ex:

Customer Behavior Features

- orders-7d
- orders-30d
- avg-order-value
- cancellation-rate

Customer Payment Features

- failed-payments-7d
- Payment-count-30d
- avg-payment

This makes features easier to:

- discover
- reuse
- manage
- version

Feature Versioning

Features can change over time.

Ex: v_1 = average over 30 days
 v_2 = weighted average over 30 days.

don't silently change the meaning of an existing feature
better:

customer-behavior- v_1

customer-behavior- v_2

Then models can explicitly depend on a particular version.

Feature-store design

Entity - a business object with a stable ID

Feature - an attribute or aggregation of that entity, always
numeric or boolean, always with a timestamp

Grain - one row per (entity, event timestamp) - same idea as
kimball fact-table grain.

Naming convention - entity-feature-aggregation-window (eg.
customer-orders-count-7d)

Feature type - batch (recomputed periodically) vs streaming
(updated on each event) vs on-demand (computed
at request time from raw inputs).

Materialization

→ means computing features and storing them in the serving
store.

Raw Data → Compute features → Materialize → Online store

Ex:

orders table → COUNT (orders in last 24h) → transactions_24h
Redis

Then inference can simply: GET customer:101

instead of calculating the feature from scratch.

Freshness SLA

→ specifies how quickly features must be updated.

Ex:

Feature: transactions - 24h

Freshness SLA: 15 minutes

Materialization Pattern

1. Batch materialization → a scheduled job (Airflow, dbt, Spark) computes features nightly or hourly. cheapest, simplest. Freshness SLA: hours to a day.

2. Streaming materialization → kafka/Flink/Kinesis update features event-by-event. Freshness SLA: seconds to minutes.

Fact Table

→ stores measurable business events/numbers.

Ex: sales

Date	Product	Customer	Quantity	Sales - Amount
2026-08-26	P101	C101	2	1000

Typical columns:

- Foreign keys → dimensions
- Measures → quantity, revenue, cost, profit, etc.

Types of Facts:

→ classified based on how they behave when aggregated using SUM.

i. Additive Fact:

→ can be added across all dimensions

Ex: Sales - Amount, Quantity - sold, Cost

Then: `SELECT SUM(Sales - Amount) FROM sales - Fact;`

ii. Semi-Additive Fact:

→ can be added across some dimensions, but NOT all dimensions.

Ex: Account - Balance, Inventory - Level

Date	Account	Balance
Monday	A1	1000
Tuesday	A1	1500

you should NOT calculate: `SELECT SUM(Balance) FROM Account - Fact;`
because: $1000 + 1500 = 2500$ doesn't represent account's actual balance.
but you can sum balances across different accounts on the same date: $A1 = 1000$ $A2 = 2000$ Total = 3000

sem

iii. Non-Additive Fact

→ cannot meaningfully added across dimensions.

Ex: Percentage, Ratio, Average, Profit Margin

Ex: Product A margin = 20%, Product B margin = 30%.

wrong: $20\% + 30\% = 50\%$.

instead:

```
SELECT SUM(Profit)/SUM(Sales) AS Profit-Margin  
FROM Sales-Fact;
```

Factless Fact Table

→ contains no numerical measures. It records that an event happened or a relationship existed.

Ex Dim: Student, Date, Class

Fact table:

Student-ID	Date-ID	Class-ID
S101	D01	C01
S102	D01	C01

There is no: Quantity, Revenue, Amount

Counting events: `SELECT COUNT(*) FROM Attendance-Fact;`

Degenerate Dimension

→ is a dimension value stored directly in the fact table, without creating a separate dimension table.

Ex: Order-ID

Suppose:

Sales-Fact

Order-ID

Product-ID

Customer-ID

Date-ID

Quantity

Sales-Amount

Order-ID is useful for identifying the transaction, but there may be no additional attributes such as: Order-Name, Order-Category, Order-City
 So we don't create: Order-Dimension

Ex: SELECT Order-ID, SUM (Sales-Amount) FROM Sales-Fact
 GROUP BY Order-ID;

Junk Dimension:

→ Combines multiple small, low-cardinality attributes into one dimension table.

instead of putting many small flags directly in the fact table:
 Is-Gift, Is-Online, Payment-Type, Order-Status, Customer-Type

Create: Junk-Dimension

Ex:

Junk-ID	Is-Gift	Is-Online	Payment-Type	Order-Status
1	Y	Y	Card	completed
2	N	Y	UPI	completed
3	N	N	Cash	Pending

Fact table:

Order-ID	Product-ID	Junk-ID	Amount
0101	P101	1	500
0102	P102	2	800

Instead of:

Fact table

Is-Gift

Is-Online

Payment-Type

Order-Status

and:

Junk-Dimension

Junk-ID

Is-Gift

Is-Online

Payment-Type

Order-Status

Fact table

Junk-ID

Role-Playing Dimensions

→ a single dimension is used multiple times in a fact table, where each usage has a different business role.

Ex: Date Dimension

One dim date can represent:

Fact - Sales

code-date-key

ship-date-key

delivery - date - key

All three keys reference the same dim-date.

dim-date

date-key

full-date⁰

month

quarters

years

Bridge Table

→ is used to handle many-to-many (M:N) relationships in a dimensional model.

Ex:

one customer can have multiple accounts, and one account can belong to multiple customers.

Instead of directly connecting them: Customer $\rightarrow$ M:N $\rightarrow$ Account
use a bridge table:

dim - customer $1:M$ $\rightarrow$ bridge - customer - account

$M=1$
↓
dim-account

Syntax:

bridge-customer-account

customer-key
account-key

Hierarchies In Dimensions

→ a hierarchy represents different levels of detail inside a dimension.

Ex: Location Hierarchy

Country → State → City → Store

This allows analysis from high-level → low level.

Common operations:

Drill-down: more detail

Year → Quarter → Month → Day

Roll-up: less detail

Day → Month → Quarter → Year

Types:

Balanced hierarchy: every branch has the same number of levels.

Country → State → City

Unbalanced hierarchy: Different branches have different levels.

Company → Department → Employee

Company → Employee

Date Dimension

→ stores information about calendar dates and is commonly connected to fact tables.

Syntax

dim-date

date-key

full-date

day

day-name

week

month

month-name

quarter

year

is-weekend

is-holiday

Ex:

date-key	full-date	month	quarter	year
20260826	2026-08-26	8	Q3	2026

why use it?

instead of calculating date information repeatedly:

YEAR (order-date)

MONTH (order-date)

the warehouse can directly use:

dim-date.year

dim-date.month

dim-date.quarter

Time Dimension

→ stores information about the time of day.

Syntax:

dim-time

time-key

hour

minute

second

am-pm

time-period

Ex:

time-key	hour	minute	am-pm
093015	9	30	AM

can also contain: Morning Afternoon Evening Night

Date vs Time

dim-date → 2026-08-26

dim-time → 09:30:15

Together: 2026-08-26 09:30:15

Mini Dimension

→ is a small dimension created by separating frequently changing attributes from a large main dimension.

Ex: dim-customer

customer-key

name

age

income

marital-status

education

city

attributes like income, marital status may change frequently.

instead of continuously updating the large customer dimension,
create a mini dimension.

structure

dim-customer

customer-key

name

city

mini-dim-key



dim-customer-profile

mini-dim-key

income-range

marital-status

education.

Data Mart

→ is a smaller, subject-specific part of a Data Warehouse, designed for a particular department or business area

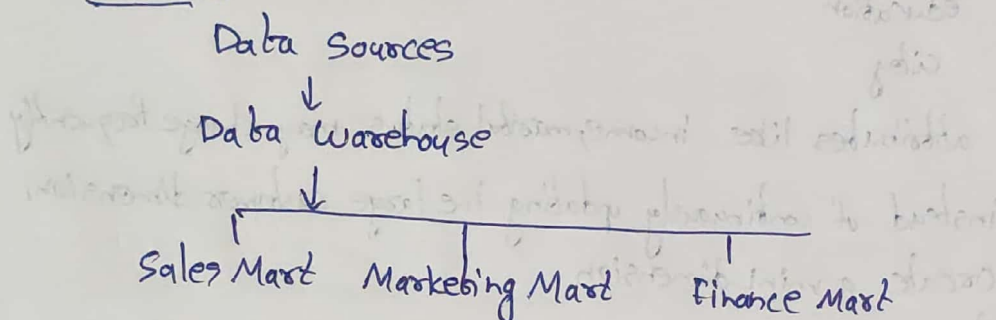
Ex:

Sales Data Mart → sales analysis

Marketing Data Mart → campaigns and customers

Finance Data Mart → revenue and expenses

Structure:



Purpose: Faster queries, Easier reporting, Department-specific data, less complexity for users.

Dimension	Dependent Mart	Independent Mart
Data source	fed from central enterprise DW	built directly from source systems
Integration	shares conformed dimensions with other marts	has its own dimension tables, own keys
cross-mart query	possible - same dimension keys and definitions	impossible or unreliable - different keys/grain
Build sequence	DW built first, mart is a filtered subset	mart built independently, fast but isolated
long-term risk	low-governed, integrated	high - becomes a permanent analytical silo
when acceptable	production standard	only for urgent one-off pilots (with a plan to integrate)

Kimball Bus Matrix

→ is a planning tool used to identify:

- Business processes
- Dimensions shared by those processes.

It helps design a consistent dimensional model

Ex: Retail

Business Process	Date	Customer	Product	store	Employee
Sales	✓	✓	✓	✓	✓
Returns	✓	✓	✓	✓	
Inventory	✓		✓	✓	
Promotions	✓		✓	✓	

here:

rows = Business processes

Columns = Dimensions

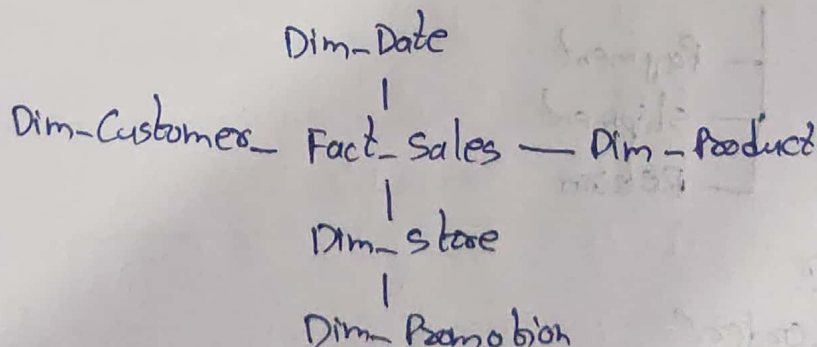
✓ = dimension is used by that process

Retail Data Modeling

→ a retail system usually deals with:

- Customers
- Products
- Stores
- Sales
- Payments
- Inventory
- Returns
- Promotions

Typical star schema



Fact-Sales

Fact-SalesC

sales-key, date-key, customer-key, product-key, store-key,
promotion-key, quantity, sales-amount, discount-amounts,
profit-amount

Dimensions

Dim-CustomerC

customer-key, customer-id, customer-name, city, segment

Dim-storeC

store-key, store-id, store-name, city, region

The fact table stores measurements, while dimensions provide context.

E-Commerce Data Modeling

- Customers • Product • Orders • Order Items • Payments
- Shipments • Returns • Reviews • Promotions

Basic model

Customer



Orders



Order Items — Product



Payment



Shipment



Return

Fact-Order

Fact-OrderC

order-key, date-key, customer-key, payment-key,
shipping-key, order-amount, discount-amount, shipping-cost

Fact-Order-Item

Fact-Order-Item C

order-item-key, order-key, date-key, customer-key,
product-key, quantity, unit-price, discount, line-amount

Cross-Domain Perspective

means analyzing data across multiple business domains
using common/contained dimensions.

Ex domains:

Sales

Inventory

Marketing

Customer Service

Finance



Enterprise Data Warehouse

"Do promotions increase sales while maintaining inventory
availability?"

we need data from:

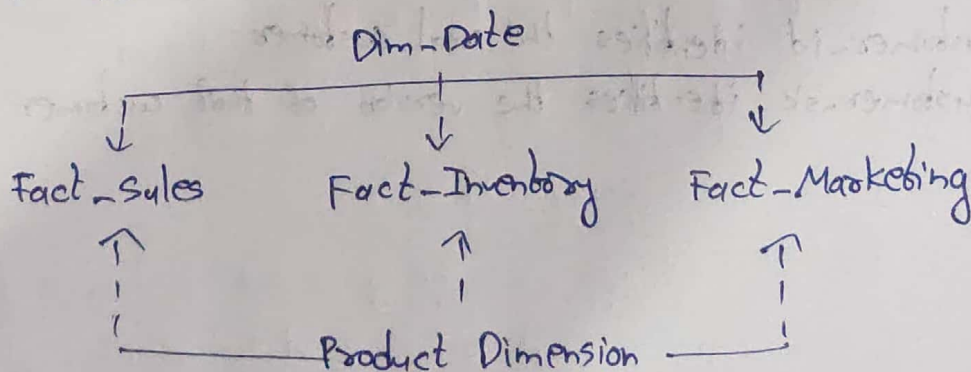
Marketing → Promotions

Sales → Sales

Inventory → Stock

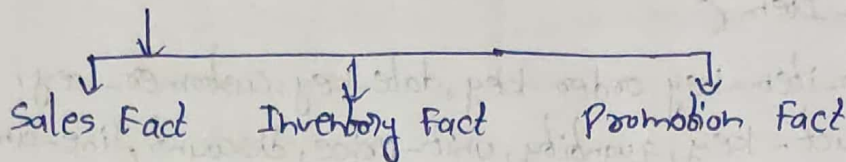
Customer → Customer

shared dimensions make this possible.



Ex:

Dim-Product



Now we can analyze:

Product + Promotion + Sales + Inventory
to understand the complete business picture.

SCD Type 2 - slowly Changing Dimension

Purpose: preserve the history of dimension changes

Dimensional structure

Customer-dim

customer-sk

-- surrogate key

customer-id

-- business/natural key

customer-name

city

start-date

end-date

is-current

Ex:

customer-sk	customer-id	city	start-date	end-date	is-current
101	C101	Hyderabad	2025-01-01	2026-05-01	No
205	C101	Bangalore	2026-05-11	999-12-31	Y

customer-id identifies the real customer

customer-sk identifies the version of that customer

Surrogate key Generation

→ Is a warehouse-generated key with no business meaning

customer-sk BIGINT GENERATED ALWAYS AS IDENTITY

(or)

customer-sk BIGSERIAL PRIMARY KEY

Ex:

customer-id = C101 -- business key

customer-sk = 205 -- warehouse key

Why use it?

- Business keys can change
- Same customer can have multiple historical versions
- Faster joins using integer keys
- Protects warehouse from source-system key changes.

SCD Type 2 using MERGE

Assume:

staging-customer

customer-id

customer-name

city

Step 1 - Expire changed records

conceptually:

```
UPDATE customer_dim d SET end_date = CURRENT_DATE -  
INTERVAL '1 day', is_current = FALSE FROM staging_customer s  
WHERE d.customer_id = s.customer_id AND d.is_current =  
TRUE AND d.city <> s.city;
```

Step 2 - Insert new version

```
INSERT INTO customer-dim (customer-id, customer-name, city,
start-date, end-date, is-current) SELECT s.customer-id, s.customer-name,
s.city, CURRENT_DATE, DATE '1999-12-31', TRUE FROM
staging-customer s LEFT JOIN customer-dim d ON
d.customer-id = s.customer-id AND d.is-current = TRUE WHERE
d.customer-id IS NULL OR d.city <> s.city;
```

MERGE is useful for combining matching / non-matching logic, but
SCD 2 usually needs careful handling of both UPDATE + INSERT. The
exact implementation differs between PostgreSQL, Snowflake, SQL
server etc.

Late Arriving Dimensions

→ A late arriving dimension occurs when a fact arrives before
its dimension record.

Ex:

Order arrives



Fact table needs customer-sk



Customer dimension hasn't arrived yet

Solution: create an interim / placeholder dimension record.

```
INSERT INTO customer-dim
(customer-id, customer-name, city, start-date, end-date,
is-current)
VALUES ('CS00', 'Unknown', 'Unknown', CURRENT_DATE,
DATE '1999-12-31', TRUE);
```

Then the fact can use the generated: customer-sk

Later, when the real customer data arrives, update the
placeholder

key idea:

Fact arrives first $\rightarrow$ create placeholder dimension
update dimension $\leftarrow$ Real dimension arrives $\leftarrow$ Load fact

slowly changing facts

$\rightarrow$ Facts normally represent events/measures, so they usually don't change like dimensions.

but some facts can change later.

Ex: Order status shipment status Account balance Subscription state

Common approaches:

Type 1 fact:

overwrite the value

UPDATE order-fact SET quantity = 10 WHERE order-id = 1001;

no history

Type 2 / accumulating fact:

keep milestones/status changes

useful for process tracking such as:

ordered $\rightarrow$ Packed $\rightarrow$ shipped $\rightarrow$ Delivered

Periodic snapshot

Take a snapshot periodically

account-balance-daily

date

account-id

balance

Ex:

Aug 29 $\rightarrow$ £ 50,000

Aug 26 $\rightarrow$ £ 52,000

Aug 27 $\rightarrow$ £ 48,000

Data quality Rules

Before loading warehouse tables, validate data.

Common rules

1. NOT NULL

2. customer-id IS NOT NULL

2. uniqueness

customer-id must be unique

3. Referential integrity

fact.customer-sk $\rightarrow$ dimension.customer-sk

4. valid ranges

quantity > 0 , price ≥ 0

5. valid dates

order-date $\leq$ CURRENT-DATE

6. Duplicate detection

```
SELECT customer-id, COUNT(*) FROM staging-customers  
GROUP BY customer-id HAVING COUNT(*) > 1;
```

7. Business rules

delivery-date $\geq$ order-date

end-date $\geq$ start-date

bad records should generally go to a:

error/reject/quarantine table

rather than silently entering the warehouse

ETL Load sequence - Dimensional warehouse

Typical order:

Extract source data

↓
Load into staging

↓
Data quality validation

↓
Load dimensions

↓
Handle SCD changes

↓
Handle late/inferred dimensions

↓
Load fact tables

↓
validate fact ↔ dimension relationships

↓
Reconcile source vs warehouse

↓
Audit/log the load

End-to-End warehouse Design

a typical dimensional data warehouse flow:

Source system



PostgreSQL / MySQL / APIs / CSV



Staging Layer



ETL / ELT



Dimensions + Facts



Data Marts



BI / Reports / Dashboards

Ex: E-commerce warehouse

Dimensions

dim-customerC

customer-key PK, customer-id, name, city, country, segment,
effective-from, effective-to, is-current

};

dim-productC

product-key PK, product-id, product-name, category, brand

};

dim-dateC

date-key PK, full-date, month, quarter, year

};

Fact

fact-salesC

sales-key PK, date-key FK, customer-key FK,
product-key FK, order-id, quantity, sales-amount,
discount, profit

};

Load sequence

1. Extract source data
2. Load staging tables
3. Validate/clean data
4. Load dimensions
5. Generate surrogate keys
6. Load fact tables
7. Run data-quality checks
8. Refresh data marts/BI

Dimensions generally need to be loaded before facts, because facts need dimension surrogate keys.

Common Data warehouse Modeling (Dimensional) Mistakes

- i. Using business keys as fact foreign keys

fact-sales $\rightarrow$ customer-key

dim-customer $\rightarrow$ customer-id

because customer-key supports SCD Type 2.

- ii. Mixing different grains

- iii. Storing descriptive attributes in fact

- iv. Not handling historical changes

- v. Incorrect fact grain

- vi. Ignoring late-arriving dimensions

Four-step Kimball Design Process

Step 1 - Select the Business Process

Ask: "what business activity are we measuring?"

Ex: Sales Orders Payments shipments Returns

Ex: Business Process = Sales

Step 2 - Declare the Grain

Ask: "what does ONE fact row represent?"

Ex: One row = one product sold in one order

Step 3 - Identify the Dimensions

Ask: "By what things do users want to analyze the process?"

For sales: Date Customer Product Store Employee Location

So: fact-sales

- dim-date
- dim-customer
- dim-product
- dim-store

Step 4 - Identify the Facts

Ask: "what numbers do we want to measure?"

Ex: quantity sales-amount discount cost profit

Then create:

fact-sales (

date-key, customer-key, product-key, store-key, quantity,
sales-amount, discount, profit

)

Mock Design Interview

step 1 - Requirements

step 2 - Business process

step 3 - Grain

step 4 - Dimensions

step 5 - Facts

step 6 - Schema

step 7 - Historical changes

step 8 - Data pipeline

step 9 - Data quality